

Reactive forms

HttpClient · live API

FormBuilder · getRawValue · CourseService · rxResource · CORS · Liya's real
TMS client

TMS client

```
this.fb.nonNullable.group({ ... });  
  
this.form.getRawValue(); // API submit  
  
@if (form.controls.studentId.touched &&  
...) { ... }
```

The browser is the first validator

Session goals

By the end of this session, you will:

- Explain why the form is the first line of defense before the TMS API
- Contrast template-driven vs Reactive Forms one CoTBE default for enrollment-scale UI
- Use FormBuilder + nonNullable.group with validators; show errors only when touched && invalid
- Submit payloads with getRawValue() when any control may be disabled (read-only IDs)
- Model backup course rows with FormArray + [formControl] in @for (spine stretch)
- Clear Session 3 traps: ngModel+formGroup, missing ReactiveFormsModule, .value vs getRawValue, markAllAsTouched

Forms are where bugs live

Your API has validation. Your database has constraints. But the first line of defense is the form in the browser. If bad data gets past the form, the user sees a cryptic 400 error from the backend instead of a helpful message next to the field they need to fix.

Angular has two approaches to forms. You are going to learn one well and acknowledge the other exists.

Template-driven vs Reactive the real difference

	Template-driven [(ngModel)]	Reactive FormGroup
Where logic lives	In the HTML template	In the TypeScript class
Unit testing	Need to render the full template	Test the form object directly no DOM
Dynamic fields	Painful	FormArray handles it cleanly
When to use	Login form with 2 fields	Everything in the TMS

CoTBE default

Reactive Forms for enrollment: conditional validation, backup rows, cross-field rules template-driven becomes unreadable fast (spine).

How Angular tracks form fields

- Three independent state axes (spine): pristine ↔ touched; valid ↔ invalid; plus the form-level valid flag
- pristine → touched when the user leaves the field; valid ↔ invalid as values change
- Show errors only when touched AND invalid never yell on first paint
- Good UX: disabled submit when form.invalid; reveal field errors after the user had a chance to type

Rule

Check both `.touched` and `.invalid` before rendering an error message spine mermaid condenses to this one gate.

Building a reactive form TypeScript

enrollment-form.component.ts (spine pattern)

```
private fb = inject(FormBuilder);

form = this.fb.nonNullable.group({

  studentId: ['', [Validators.required, Validators.pattern('^STU-[0-9]{4}$')]],

  courseId: ['', Validators.required],

  term: ['Fall 2026', Validators.required],

  notes: ['']

});
```

- nonNullable values typed as string, not string | null (spine: fewer null checks in every handler)
- ['', validators] default value first, validators second (spine / workbook pattern)
- Without nonNullable, you spend half your time asking "is this field null?" for no benefit (spine)

Building a reactive form template

Template (spine condensed)

```
<form [formGroup]="form" (ngSubmit)="submit()">

  <label for="studentId">Student ID</label>

  <input id="studentId" formControlName="studentId" placeholder="STU-1234" />

  @if (form.controls.studentId.touched && form.controls.studentId.invalid) {

    <span class="error">Enter a valid Student ID (format: STU-0000)</span>

  }

  <button type="submit" [disabled]="form.invalid">Confirm Enrollment</button>

</form>
```

- [formGroup] binds the <form> to the TS object; formControlName links each input to a key
- Submit runs submit(); [disabled]="form.invalid" blocks hopeless posts while fields are still pristine

The getRawValue() rule

.value BAD with disabled fields

- `studentId.disable()` for read-only display
- `const payload = this.form.value;`
- Student ID missing from JSON silent drop
- .NET returns 400 looks like API bug

getRawValue() GOOD for API

- Includes disabled controls in the object
- `const payload = this.form.getRawValue();`
- Full enrollment payload honest POST
- Make it the habit for TMS submits (spine)

Dynamic fields FormArray (stretch)

Why FormArray (spine)

TMS enrollment: 1–5 backup course choices. You cannot hardcode five inputs you do not know how many Liya needs until she clicks Add.

TypeScript (spine)

```
backupCourses = this.fb.array<FormControl<string>>([]);

addBackup() {
  this.backupCourses.push(
    this.fb.control('', { nullable: true, validators: Validators.required })
  );
}

removeBackup(index: number) { this.backupCourses.removeAt(index); }
```

Template (spine)

```
@for (backup of backupCourses.controls; track $index) {

  <div class="backup-row">

    <input [formControl]="backup" [placeholder]="'Backup ' + ($index + 1)" />
    <button type="button" (click)="removeBackup($index)">Remove</button>
  </div>
}
<button type="button" (click)="addBackup()">Add Backup Course</button>
```

Common mistakes Session 3

Mistake	What you see	How to fix
Using <code>[(ngModel)]</code> with <code>[formGroup]</code>	Angular throws: "Cannot use ngModel in a formGroup"	Pick one strategy. For this form, use only <code>formControlName</code> .
Validation messages always visible	Error text appears before user touches the field	Check both <code>.invalid</code> AND <code>.touched</code> before showing errors
Missing <code>ReactiveFormsModule</code>	<code>formGroup</code> is an unknown directive	Add <code>ReactiveFormsModule</code> to the component's imports array
Using <code>.value</code> on forms with disabled fields	Backend returns 400 missing required field	Use <code>.getRawValue()</code> instead
<code>markAllAsTouched()</code> not called on submit	User submits invalid form, sees no error messages	Call <code>this.form.markAllAsTouched()</code> before showing errors

Part A summary Reactive Forms

- Reactive Forms own TMS enrollment complexity in TypeScript, not in template spaghetti
- touched && invalid professional error timing
- getRawValue() disabled fields still ship to .NET
- FormArray + [formControl] backup course rows without hardcoding five inputs

Then Part B (same thematic session)

HttpClient, CourseService, rxResource retire mock arrays and prove GET/POST against the M6/M7 API.

Same universe now they talk



Part B goals HttpClient and the live API

By the end of Part B, you will:

- Retire hardcoded course arrays read live M6/M7 data the way Liya will in production
- Keep components URL-free CourseService owns /api/courses and HttpClient typing
- Prefer rxResource for GET lists loading / error / value signals without manual subscribe cleanup
- Map three template states to coursesResource.isLoading / .error / .value (spine)
- Triage CORS between localhost:4200 and localhost:5001 fix on the API with AddCors / UseCors for the Angular dev origin
- Use .subscribe() + takeUntilDestroyed(destroyRef) for mutations with navigation side-effects

Mock data is over

For five labs, you used hardcoded arrays. That was necessary you were learning Angular's component model. But a hardcoded array cannot tell you if a course filled up five minutes ago. It cannot show a new course the admin just created.

From this point forward, the data comes from the .NET API you built in M6 and M7. Your Angular app becomes a real client.

Rule: components never know URLs

Components should not contain localhost:5001/api/courses they say "Give me the courses." A service owns paths and HttpClient (spine).

course.service.ts (spine trimmed comments)

```
// import { map } from "rxjs/operators";

@Injectable({ providedIn: "root" })
export class CourseService {

  private http = inject(HttpClient);

  private baseUrl = "/api/courses";

  getAll() {
    return this.http
      .get<PagedResponse<Course>>(this.baseUrl, {
        params: { page: "1", pageSize: "50" },
      })
      .pipe(map((p) => p.items));
  }

  getById(id: string) {
    return this.http.get<CourseDetail>(`${this.baseUrl}/${id}`);
  }
}
```

- providedIn: 'root' one shared instance (think AddSingleton<T>()); not listed in component imports[]
- GET /api/courses returns PagedResponse map .items[] for the grid; detail uses CourseDetail (spine)
- URL change → one file; mock the service for tests; swap backend without touching templates (spine)

The problem with .subscribe() (data loads)

Anti-pattern

```
// DON'T DO THIS for data loading (spine)

ngOnInit() {

  this.api.getAll().subscribe(data => {

    this.courses = data;

  });

}
```

- Subscription survives navigation the callback can outlive the component (spine)
- Dead subscriptions accumulate after hours of use the browser gets sluggish and can crash (spine)
- takeUntilDestroyed() fixes leaks rxResource removes the whole class of bug for reads (spine)

rxResource wraps an HTTP Observable into managed signals - subscribe, unsubscribe, loading, and error handled for you (spine).

rxResource the modern default

Spine definition

```
export class CourseListComponent {  
  private api = inject(CourseService);  
  
  coursesResource = rxResource({  
    loader: () => this.api.getAll(),  
  });  
}
```

- `coursesResource.value()` the data (once loaded) (spine)
- `coursesResource.isLoading()` true while the HTTP request is in flight
- `coursesResource.error()` the error if the request fails
- No `.subscribe()`, no `.unsubscribe()`, no `takeUntilDestroyed()` resource owns lifecycle (spine)

Binding the resource to the template

Template (spine)

```
@if (coursesResource.isLoading()) {  
    <div class="spinner">Fetching courses from the server...</div>  
} @else if (coursesResource.error()) {  
    <div class="error">Could not reach the TMS server. Make sure your .NET API is running.</div>  
} @else {  
    @for (course of coursesResource.value()!; track course.id) {  
        <tms-course-card [course]="course" />  
    }  
}
```

- ! on .value() in @else TS non-null assertion: loading and error are false here (spine comments)
- Three states user always knows fetching vs failure vs cards (spine)

CORS you will hit this

When Angular on localhost:4200 tries to call .NET on localhost:5001, the browser may block the response (spine).

Access to XMLHttpRequest at 'https://localhost:5001/api/courses' from origin 'http://localhost:4200' has been blocked by CORS policy.

This is not an Angular bug. It is not a .NET bug. It is the browser defending the user from malicious cross-origin requests.

Program.cs (spine backend fix)

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowAngular", policy =>
    {
        policy.WithOrigins("http://localhost:4200")
               .AllowAnyHeader()
               .AllowAnyMethod());
    });
});

app.UseCors("AllowAngular");
```

When .subscribe() is still correct

Pattern (spine shortened)

```
// Mutations: navigate / toast on success (spine)

enroll() {

  this.api.create(this.form.getRawValue())

    .pipe(takeUntilDestroyed(this.destroyRef))

    .subscribe({

      next: (result) => this.router.navigate(['/enrollments', result.id]),

      error: (err) => this.errorMessage.set(err.message)

    });

}
```

- private destroyRef = inject(DestroyRef); pair every mutation subscribe with takeUntilDestroyed (spine)
- rxResource = reads; subscribe + cleanup = writes with side-effects

Read vs write patterns (spine table)

Pattern	Use for	Why
rxResource()	GET requests (lists, lookups, dashboards)	Auto-manages lifecycle, loading, error states
.subscribe() + takeUntilDestroyed()	POST / PUT / DELETE (mutations with side-effects)	Need specific logic on success/failure
toSignal()	Static one-time lookups (app config, feature flags)	Simpler than rxResource when you never need to refresh

Common mistakes HttpClient and rxResource

Mistake	What you see	How to fix
Missing provideHttpClient()	NullInjectorError: No provider for HttpClient	Add provideHttpClient() to app.config.ts
CORS error in browser console	Red error: "blocked by CORS policy"	Fix on the .NET side: add CORS middleware
.value without ()	Template shows signal wrapper, not data	Use coursesResource.value() with parentheses
Nested .subscribe() calls	Code deeply indented, requests fire sequentially	Use `switchMap` or `forkJoin` or wait for M9
.subscribe() without cleanup	Memory leaks after navigating away	Use `takeUntilDestroyed(this.destroyRef)`

What comes next Module 9

Your Angular app renders data, handles forms, and talks to the backend. But it has a gap: if Liya sees 12 open seats and another student enrolls from a different device, Liya still sees 12 until she refreshes. The data is not live.

M9 introduces shared state (NgRx SignalStore) and real-time updates (SignalR) to close that gap.

Module 8 complete

TMS client: shell, composition, routing, reactive forms, live API M9 adds shared state and SignalR for seats that stay honest.